

Project Marking Rubric: Stages 2-4

COMP 2659 – Fall 2024

Group: ?

Mark: ? / 100

Important Information for Students

The project is worth 27% of your final course grade, with stages 2-4 comprising the second 7%. The overall project grade, including the mark for stages 2-4, will not be finalized until after the December due date. The mark for stages 2-4 indicated here is *provisional*: groups are eligible to receive a higher mark if they make improvements to these stages of the final submitted game. That is, the mark for stages 2-4 may ultimately be higher. The final mark for stages 2-4 mark will not normally be lower, except as indicated below.

Penalties to the final project grade that affect the provisional mark for stages 2-4 will only be applied if the group did not submit a reasonably complete set of working and testable model 2-4 code by the October 25 deadline.

Please be reminded that you are required to keep your stage 1 game specification up to date as you develop the remainder of the project. If you make changes to your game concept, ensure that these are reflected in the document.

Please see the course outline and project requirements documents for more information.

General Instructor Comments

?

Rubric

1) Miscellaneous

/ 25

Project and test driver builds have been automated using a make file	/ 10
Good code and documentation practices have been followed in accordance with the posted requirements • E.g., each function has one clear, cohesive purpose • E.g., algorithm design is clear • E.g., functions are of reasonable length (max. 25-30 lines and often shorter) and complexity; significant sub-tasks are delegated to sub-functions • E.g., code is appropriately self-documenting and clearly formatted • E.g., documentation is used effectively to explain non-obvious design and implementation details	/ 15

2) Stage 2: Raster Graphics Library

/ 25

An appropriate set of low-level plotting routines has been provided: • A generic (i.e., game-agnostic) set of basic plotting routines sufficient for supporting game rendering in stage 4 • The raster module is fully decoupled from the implementation of the model and rendering stages • A frame buffer pointer is supplied as a parameter to each plotting function • Implemented without library or operating system support; each routine plots directly and correctly to the given frame buffer • Deals correctly with arbitrary plotting locations and screen boundaries (to the extent this is necessary for the game)	/ 9
A robust and systematic raster module test driver provides high confidence that the plotting routines work correctly across a wide variety of automated test cases	/ 8
Each routine's implementation is reasonably clear and efficient	/ 4
The <code>raster.c</code> module and its header file have been set up correctly	/ 4

3) Stage 3: Model

/ 25

Model data structures appropriately represent all game objects and their properties (types defined in <code>model.h</code>)	/ 6
Object-manipulating functions appropriately implement object state querying and modification, implementing all necessary object behaviours (prototyped in <code>model.h</code> and implemented in <code>model.c</code>) Implementations are decoupled from plotting and rendering Functions trigger condition-based event handlers where appropriate	/ 7
The <code>events.c</code> module contains functions (prototyped in <code>events.h</code>) for appropriately handling each type of event – one function for each possible asynchronous (user input-triggered), synchronous (clock-triggered), and conditional (object-behaviour-triggered) event type Each of these implementations should be fairly short and simple at this point, decoupled from object state manipulation, plotting, and rendering. Each merely invokes the appropriate object-manipulation function(s)	/ 6
Robust and systematic model/event test drivers provide high confidence that the routines above work correctly across a wide variety of automated test cases	/ 6

4) Stage 4: Renderer

/ 25

Object-rendering functions defined in <code>renderer.c</code> invoke <code>raster</code> module plotting functions to render an image of the given object A master render function uses the above to render a complete frame of animation of the given model (game world snapshot) to the given framebuffer As appropriate to the game, the necessary bitmap assets have been defined as global constant arrays, ideally in a separately compiled <code>bitmaps</code> module Note: it is not necessary to have implemented stage 4b (minimizing screen plotting) at this time	/ 15
Robust and systematic renderer test drivers provide high confidence that the routines above work correctly across a wide variety of automated test cases, rendering multiple static game world snapshots	/ 10